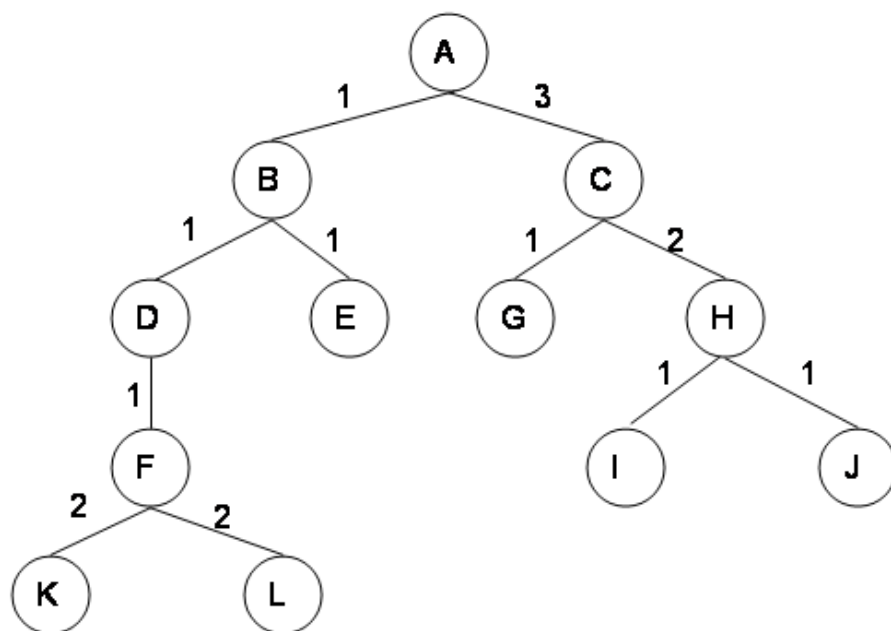


6.034 Quiz 1, Spring 2004 — Solutions

Open Book, Open Notes

1 Tree Search (12 points)

Consider the tree shown below. The numbers on the arcs are the arc lengths.

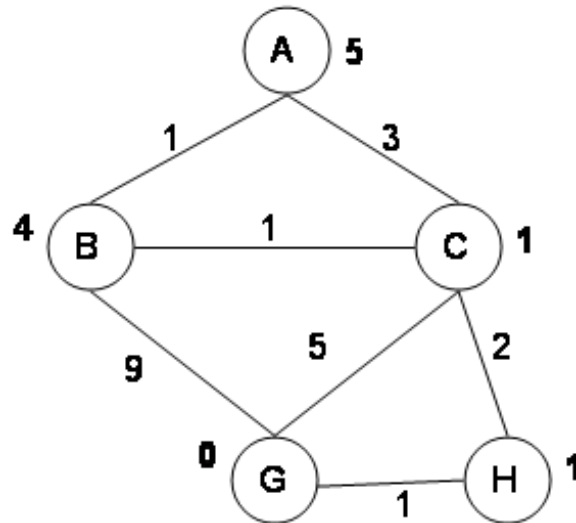


Assume that the nodes are expanded in alphabetical order when no other order is specified by the search, and that the goal is state *G*. No visited or expanded lists are used. What order would the states be expanded by each type of search? Stop when you expand *G*. Write only the sequence of states expanded by each search.

Search Type	List of states
Breadth First	A B C D E G
Depth First	A B D F K L E C G
Progressive Deepening Search	A A B C A B D E C G
Uniform Cost Search	A B D E C F G

2 Graph Search (10 points)

Consider the graph shown below where the numbers on the links are link costs and the numbers next to the states are heuristic estimates. Note that the arcs are undirected. Let A be the start state and G be the goal state.



Simulate A* search with a strict expanded list on this graph. At each step, show the path to the state of the node that's being expanded, the length of that path, the total estimated cost of the path (actual + heuristic), and the current value of the expanded list (as a list of states). You are welcome to use scratch paper or the back of the exam pages to simulate the search. However, please transcribe (only) the information requested into the table given below.

Path to State Expanded	Length of Path	Total Estimated Cost	Expanded List
A	0	5	(A)
C-A	3	4	(C A)
B-A	1	5	(B C A)
H-C-A	5	6	(H B C A)
G-H-C-A	6	6	(G H B C A)

3 Heuristics and A* (8 points)

1. Is the heuristic given in Problem 2 admissible? Explain.

Yes. The heuristic is admissible because it is less than or equal to the actual shortest distance to the goal.

2. Is the heuristic given in Problem 2 consistent? Explain.

No, the heuristic is not consistent. There are two places in the graph where consistency fails. One is between A and C where the drop in heuristic is 4, but the path length is only 3. The other is between B and C where the drop in heuristic is 3 but the path length is only 1.

3. Did the A* algorithm with strict expanded list find the optimal path in the previous example? If it did find the optimal path, explain why you would expect that. If it didn't find the optimal path, explain why you would expect that and give a simple (specific) change of state values of the heuristic that would be sufficient to get the correct behavior.

A with a strict expanded list will not find the shortest path (which is ABCHG with cost 5). This is because the heuristic is not consistent. We can make the heuristic consistent by changing its value at C to be 3. There are other valid ways to make the graph consistent (change $h(B)$ to 2 and $h(A)$ to 3, for example) and those were right as well.*

4 Search problem formulation (10 points)

A Mars rover has to leave the lander, collect rock samples from three places (in any order) and return to the lander.

Assume that it has a navigation module that can take it directly from any place of interest to any other place of interest. So it has primitive actions *go-to-lander*, *go-to-rock-1*, *go-to-rock-2*, and *go-to-rock-3*.

We know the time it takes to traverse between each pair of special locations. Our goal is to find a sequence of actions that will perform this task in the shortest amount of time.

1. Formulate this problem as a search problem by specifying the state space, initial state, path-cost function, and goal test. Try to be sure that the state space is detailed enough to support solving the problem, but not redundant.

- States: $\langle \textit{current-location}, \textit{have-rock1?}, \textit{have-rock2?}, \textit{have-rock3?} \rangle$

These are state *variables*. The variable *current-location* ranges over the set $\{\textit{lander}, \textit{rock1}, \textit{rock2}, \textit{rock3}\}$. The other variables are binary.

- Initial state: $\langle \textit{lander}, \textit{no}, \textit{no}, \textit{no} \rangle$
- Path cost: sum of arc costs; arc cost = distance between locations
- Goal test: $\langle \textit{lander}, \textit{yes}, \textit{yes}, \textit{yes} \rangle$

2. Say what search technique would be most appropriate, and why.

We want a shortest path, so we need UCS or A. We might as well use A*, since it will probably be faster and there's a reasonable heuristic available.*

3. One possible heuristic evaluation function for a state would be the distance back to the lander from the location of the state; this is clearly admissible. What would be a more powerful, but still admissible, heuristic for this problem? (Don't worry about whether it's consistent or not.)

*This should have read "One possible heuristic evaluation function for a state would be the **amount of time** required for the robot to go back to the lander from the location of the state..."*

So, because of the typo, we gave everyone a free two points on this problem.

The answer we had in mind was the maximum, over uncollected rocks r , of the time to get from the current location to r , and the time to get from r to the lander.

5 CSP (17 points)

Let's look at the problem of scheduling programs on a set of computers as a constraint satisfaction problem.

We have a set of programs (jobs) J_i to schedule on a set of computers (machines) M_j . Each job has a maximum running time R_i . We will assume that jobs (on any machines) can only be started at some pre-specified times T_k . Also, there's a T_{max} time by which all the jobs must be finished running; that is, start time + running time is less than or equal to max time. For now, we assume that any machine can execute any job.

Let's assume that we attack the problem by using the jobs as variables and using values that are each a pair (M_j, T_k) . Here is a simple example.

- Running time of J_1 is $R_1 = 2$
 - Running time of J_2 is $R_2 = 4$
 - Running time of J_3 is $R_3 = 3$
 - Running time of J_4 is $R_4 = 3$
 - Starting times $T_k = \{1, 2, 3, 4, 5\}$
 - Two available machines M_1 and M_2 .
 - The max time is $T_{max} = 7$.
 - An assignment would look like $J_1 = (M_2, 2)$, that is, run job J_1 on machine M_2 starting at time 2.
1. What are the constraints for this type of CSP problem? Write a boolean expression (using logical connectives and arithmetic operations) that must be satisfied by the assignments to each pair of variables. In particular:
- J_i with value (M_j, T_k)
 - J_m with value (M_n, T_p)

There is a unary constraint on legal values for a single variable: $T_k + R_i \leq T_{max}$. This is not a binary constraint on pairs of values.

The binary constraint is the one that says that jobs on the same machines must not overlap in time. It can be expressed as:

$$M_j = M_n \rightarrow T_k + R_i \leq T_p \vee T_p + R_m \leq T_k$$

So, either the machines are different or the times don't overlap.

2. Write down a complete valid solution to the example problem above.

- $J_1 = (M_1, 1)$
- $J_2 = (M_1, 3)$
- $J_3 = (M_2, 1)$
- $J_4 = (M_2, 4)$

Several other answers are also legal.

3. Which variable would be chosen first if we did BT-FC with dynamic ordering of variables (most constrained)? Why?

J_2 would be chosen since it has the smallest domain of legal values. That job since it takes 4 time steps can only be started at times less than or equal to 3 so that it will finish before $T_{max} = 7$.

4. If we do constraint propagation in the initial state of the example problem, what domain values (if any) are eliminated? Explain.

If one assumes that domain values inconsistent with the unary (T_{max}) constraint have been eliminated from the domains before constraint propagation, then no further domain values are eliminated. We can always run a pair of jobs on different machines and so the binary constraints do not reduce the domain further. Many people assumed that the unary constraints were checked during propagation and we allowed that.

5. If we set $J_2 = (M_1, 1)$, what domain values are still legal after forward checking?

- $J_1 \in (M_1, 5), (M_2, t) t \in \{1, \dots, 5\}$
- $J_2 \in (M_1, 1)$
- $J_3 \in (M_2, t) t \in \{1, \dots, 4\}$
- $J_4 \in (M_2, t) t \in \{1, \dots, 4\}$

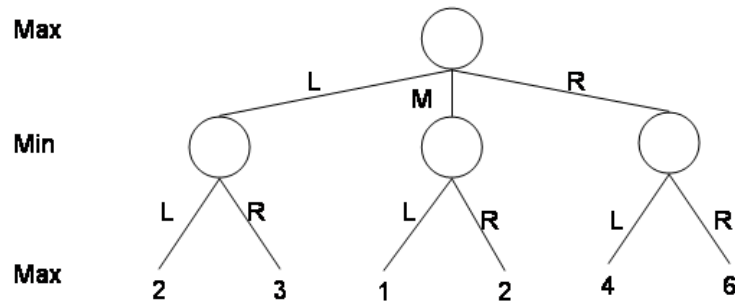
6. We could have formulated this problem using the machines M_j as the variables. What would the values be in this formulation, assuming you have N machines and have K jobs to schedule?

A value would be a complete schedule for each machine, that is, a list of all the jobs to run on the machine. One could also specify the starting times of each job but that's redundant, since the running time could be used.

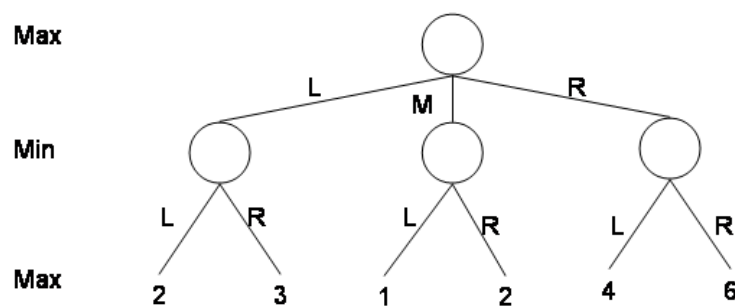
7. What are some disadvantages of this formulation (using machines as variables)? *There would be an very large number of possible values in the domain of each variable (every way of splitting K jobs among M machines so that the sum of the running times is less than T_{max}).*

6 Game Search (10 points)

Consider the game tree shown below. The top node is a max node. The labels on the arcs are the moves. The numbers in the bottom layer are the values of the different outcomes of the game to the max player.



1. What is the value of the game to the max player?
4
2. What first move should the max player make?
R
3. Assuming the max player makes that move, what is the best next move for the min player, assuming that this is the entire game tree?
L
4. Using alpha-beta pruning, consider the nodes from **right to left**, which nodes are cut off? Circle the nodes that are not examined.



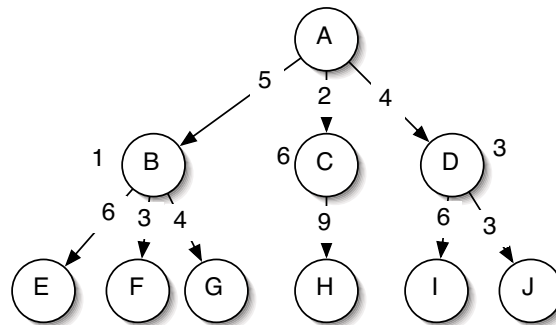
The nodes that are not examined are the left-most node labeled “2” and the node labeled “1.”

6.034 Quiz 1, Spring 2003: Solutions v. 1.1

Open Book, Open Notes

1 Tree Search (10 points)

Consider the tree shown below. The numbers on the arcs are the arc lengths; the numbers near states B, C, and D are the heuristic estimates; all other states have a heuristic estimate of 0.

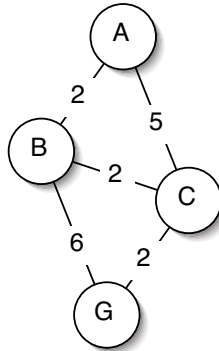


Assume that the children of a node are expanded in alphabetical order when no other order is specified by the search, and that the goal is state J. No visited or expanded lists are used. What order would the states be expanded by each type of search. Write only the sequence of states expanded by each search.

Search Type	List of states
Breadth First	A B C D E F G H I J
Depth First	A B E F G C H D I J
Progressive Deepening Search	A A B C D A B E F G C H D I J
Best-First Search	A B E F G D I J
A* Search	A B D J

2 Graph Search (8 points)

Consider the graph shown below. Note that the arcs are undirected. Let A be the start state and G be the goal state.



Simulate uniform cost search with a strict expanded list on this graph. At each step, show the state of the node that's being expanded, the length of that path, and the current value of the expanded list (as a list of states).

State Expanded	Length of Path	Expanded List
A	0	(A)
B	2	(B A)
C	4	(C B A)
G	6	(G C B A)

3 A* Algorithm (12 points)

1. Let's consider three elements in the design of the A* algorithm:

- The heuristic, where the choices are:
 - **arbitrary** heuristic
 - **admissible** heuristic
 - **consistent** heuristic
- History:
 - **none**
 - **visited** list
 - **strict** expanded list
 - **non-strict** expanded list
- Pathmax
 - **Use** pathmax
 - **Don't use** pathmax

In the table below, indicate all the combinations that *guarantee* that A* will find an optimal path. Not all rows have to be filled. If multiple values works for any of Heuristic, History and Pathmax, independent of the other choices, you can write the multiple values in one row. So

Heuristic	History	Pathmax
A,B	C	D,E

can be used to represent all of: A,C,D; A,C,E; B,C,D; and B,C,E.

Heuristic	History	Pathmax
Admissible	None, Non-Strict	Use, Don't Use
Consistent	None, Non-Strict, Strict	Use, Don't Use

2. In the network of problem 2, assume you are given the following heuristic values:

$$A = 5; B = 4; C = 0; G = 0$$

Is this heuristic:

- Admissible? ☒ Yes No
- Consistent? Yes ☒ No

Justify your answer very briefly.

It is admissible because it is always less than the length of the shortest path. It is not consistent because the difference between the heuristic values at B and C is 4, which is greater than the arc-length of 2.

3. With the heuristic above will A* using a strict expanded list find the optimal path?

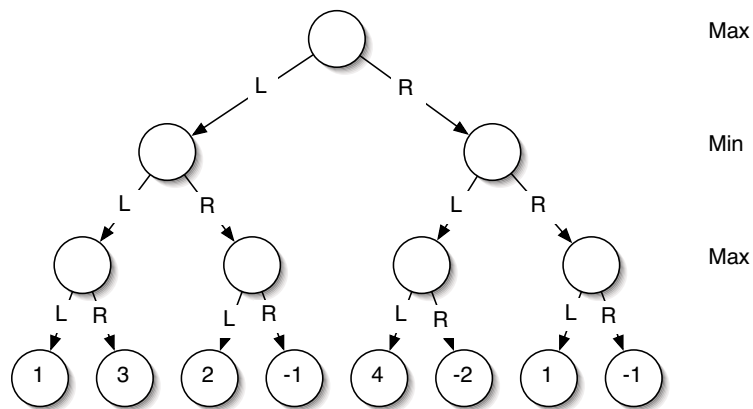
Yes ☒ No

Justify your answer very briefly.

We will visit C first from A with estimated cost of 5, and because it's on the expanded list, even when we later find a path to C with estimated cost of 4, we won't expand it again.

4 Game Search (5 points)

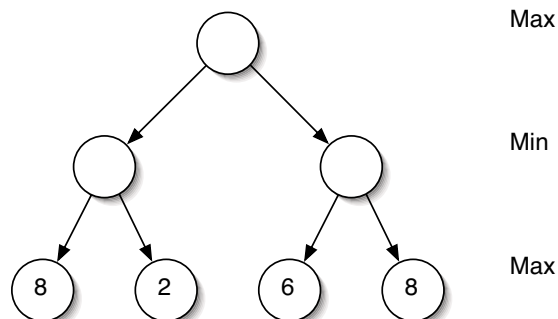
Consider the game tree shown below. Assume the top node is a max node. The labels on the arcs are the moves. The numbers in the bottom layer are the values of the different outcomes of the game to the max player.



1. What is the value of the game to the max player? **2**
2. What first move should the max player make? **L**
3. Assuming the max player makes that move, what is the best next move for the min player, assuming that this is the entire game tree? **R**

5 Alpha-Beta Pruning (5 points)

In the following game tree, are there any alpha-beta cutoffs?



- Consider the nodes from left to right, which nodes are cutoff? Circle the nodes that are not examined and label them with L. **None**.
- Consider the nodes from right to left, which nodes are cutoff? Circle the nodes that are not examined and label them with R. **The leftmost 8 node**.

6 CSP Methods (15 points)

Let's consider some combinations of CSP methods. For each of the combinations described below say very briefly whether:

1. It would be **well-defined** to combine them, in the sense that none of the implementation assumptions of the methods as we defined them are violated in the combination.
2. It could be **useful**, that is, one would expect improved performance (over using only the first method mentioned), at least in some problems. Improved performance could be either from being better able to solve problems or improved efficiency (indicate which).

In each case, circle Yes or No for each of Well-Defined? and Useful? and give a very brief explanation of your answers.

Warning: Please pay careful attention to the definition of the methods being combined, we are referring to the original definition of the methods – in isolation. Almost any idea can be made to work with any other idea with sufficient creativity - but that's not what we are looking for in this problem.

- Full constraint propagation (CP) followed by pure backtracking (BT).

1. Well-Defined? ☒ **Yes** No

2. Useful? ☒ **Yes** No

After full CP, there may still be multiple solutions, and BT will choose one.

- Full constraint propagation (CP) combined with forward checking (FC).

1. Well-Defined? Yes ☒ **No**

2. Useful? Yes No

This doesn't make sense; you still need to do some kind of search. Having done CP, FC won't rule out any more options, and you're may be left with multiple possible solutions.

- Pure backtracking (BT) combined with dynamic variable (most constrained) and value ordering (least constraining).

1. Well-Defined? Yes ☐ No ☒

2. Useful? Yes ☐ No ☐

Dynamic variable and value ordering only make sense if you're doing FC to discover changes in legal variable domains.

- Min-conflict-hill-climb (MC) combined with dynamic variable (most constrained) and value ordering (least constraining).

1. Well-Defined? Yes ☐ No ☒

2. Useful? Yes ☐ No ☐

MC always works with a complete assignment of values to variables.

- Pure backtracking (BT) combined with full constraint propagation (CP) after each tentative assignment.

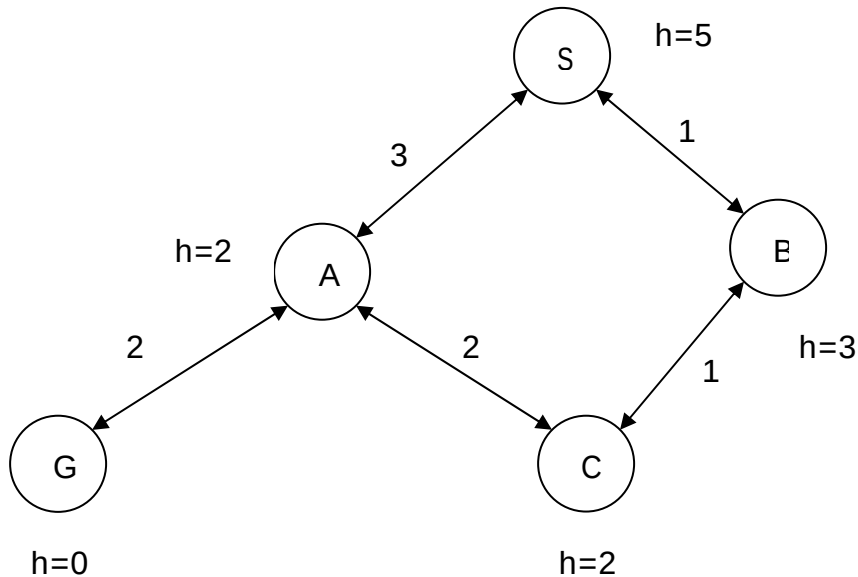
1. Well-Defined? Yes ☒ No ☐

2. Useful? Yes ☒ No ☐

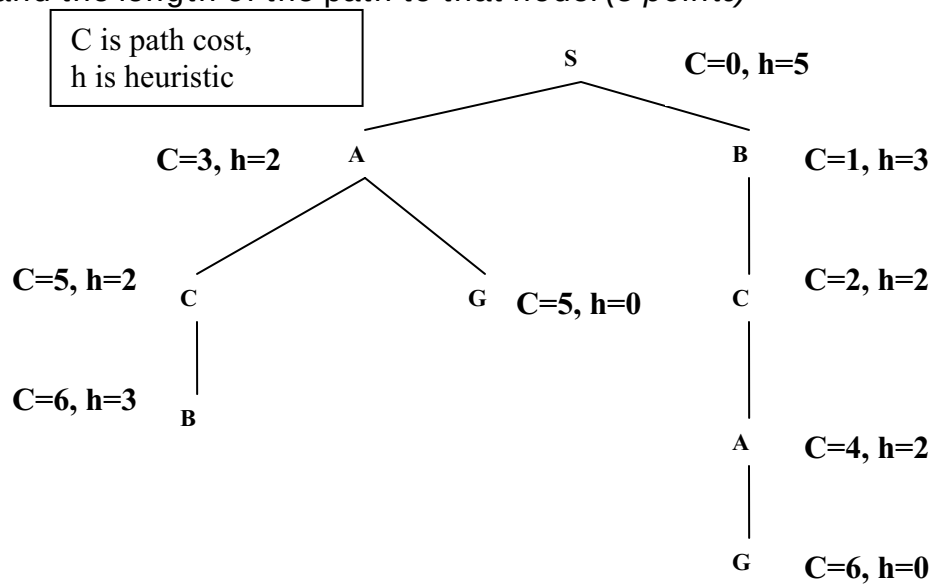
Although full CP is expensive, uninformed BT can be even worse; so, in some cases, this is an improvement.

Problem 1 – Search (30 points)

Below is a graph to be searched (starting at S and ending at G). Link/edge costs are shown as well as heuristic estimates at the states. You may not need all the information for every search.



Draw the complete search tree for this graph. Label each node in the tree with the cost of the path to that node and the heuristic cost at that node. When you need to refer to a node, use the name of the corresponding state and the length of the path to that node. (5 points)



For each of the searches below, just **give a list of node names** (state name, length of path) drawn from the tree above. **Break ties using alphabetical order.** (2 points each)

1. Perform a depth-first search using a visited list. Assume children of a state are ordered in alphabetical order. Show the sequence of nodes that are expanded by the search.

S0, A3, C5, G5 note that B6 is not expanded because B is on visited list (placed there when S0 was expanded).

2. Perform a best-first (greedy search) without a visited or expanded list. Show the sequence of nodes that are expanded by the search.

S0 (h=5), A3(h=2), G5(h=0)

3. Perform a Uniform Cost Search without a visited or expanded list. Show the sequence of nodes that are expanded by the search.

S0, B1, C2, A3, A4, C5, G5 note that nodes are ordered first by cost then alphabetically when tied for cost.

4. Perform an A* search (no pathmax) without an expanded list. Show the sequence of nodes that are expanded by the search.

S0(0+5), B1(1+3), C2(2+2), A3(3+2), G5(5+0)

Is the heuristic in this example

1. admissible? Yes

2. consistent? No

Justify your answer, briefly. (3 points)

All the h values are less than or equal to actual path cost to the goal and so the heuristic is admissible.

The heuristic drops from 5 at S to 3 at B while the path cost between S and B is only 1, and so the heuristic is not consistent.

For each of the following situations, pick the search that is most appropriate (be specific about visited and expanded list). Give a one sentence reason why you picked it. If you write a paragraph, we will not read it.

1. We have a very large search space with a large branching factor and with possibly infinite paths. We have no heuristic. We want to find paths to the goal with minimum numbers of state.

Iterative deepening is the best choice, it uses little memory (like DFS) but guarantees finding the path with minimum number of states (like BFS).

2. We have a space with a manageable number of states but lots of cycles in the state graph. We have links of varying costs but no heuristic and we want to find shortest paths.

Uniform Cost Search with a strict expanded list is the best choice, it guarantees finding shortest paths and the expanded list limits the cost to a function of the number of states, which is reasonable in this case. Recall that a visited list will interfere with the correct operation of UCS.

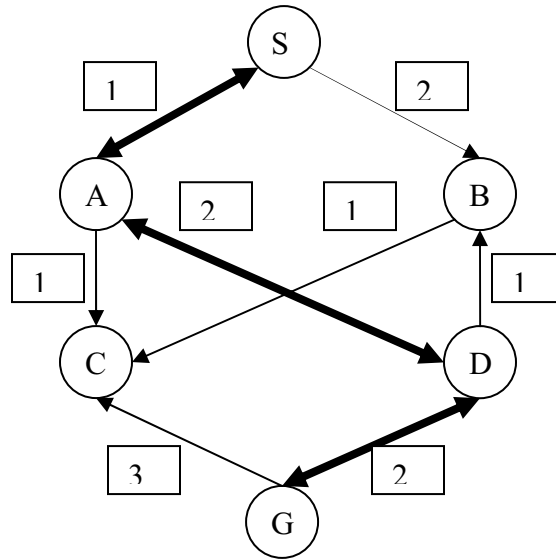
3. Our search space is a tree of fixed depth and all the goals are the leaves of the tree. We have a heuristic and we want to find any goal as quickly as possible.

This has a typo which makes it ambiguous. If you read it as "all the leaves are goals", then depth-first search is the best choice (gets to the leaves fastest). If you read it as "all the goals are at the leaves", then the best choice is a greedy search (best first), which uses the heuristic to guide you to the part of the tree with the goals. In neither case is a visited or expanded list advisable since we are searching a tree (no loops).

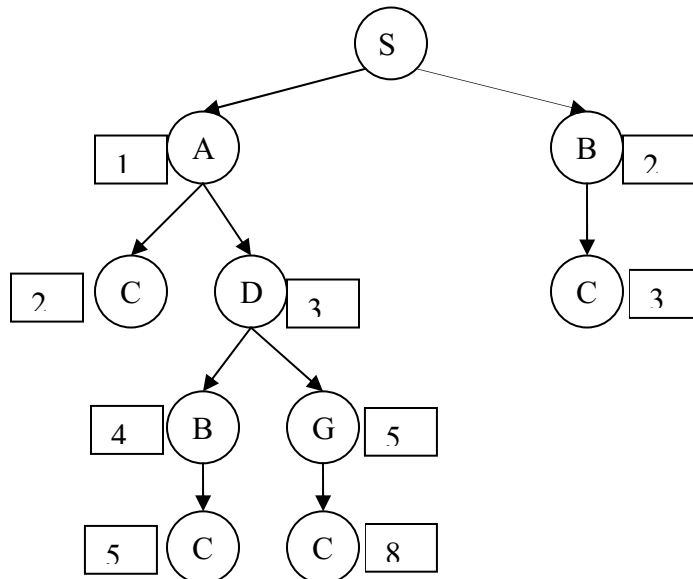
4. We have a space with a manageable number of states but lots of cycles in the state graph. We have links of varying costs and an admissible heuristic and we want to find shortest paths.

This calls for A and a non-strict expanded list and, since we don't know that the heuristic is consistent, using pathmax. This allows us to use all the information we have and to avoid the extra cost due to cycles.*

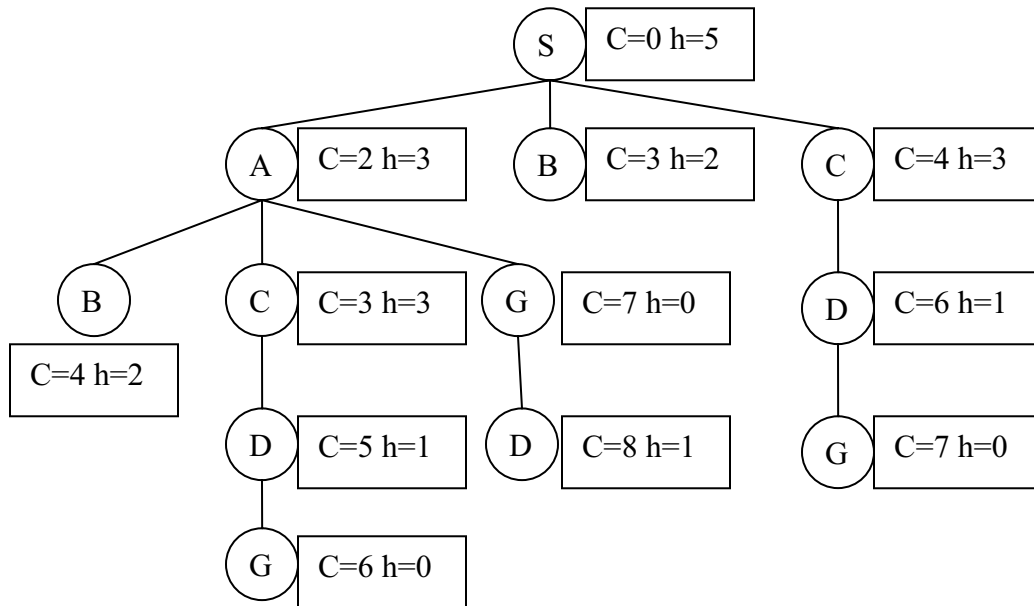
Problem 1: Search (25 points)



A. Construct the search tree for the graph above, indicate the path length to each node. The numbers shown above are link lengths. Pay careful attention to the arrows; some are bi-directional (shown thick) while some are uni-directional.



B. Using the following search tree, perform the searches indicated below (always from S to G). Each node shows both the total path cost to the node as well as the heuristic value for the corresponding state.



For each of the searches below, write the sequence of nodes **expanded** by the search. Specify a node by writing the name of the state and the length of the path (C above), e.g. S0, B3, etc. Break ties using alphabetical order.

1. Depth First Search (no visited list)

S0, A2, B4, C3, D5, G6

2. Breadth First Search (with visited list)

S0, A2, B3, C4, G7

3. Uniform Cost Search (with strict expanded list)

S0, A2, B3, C3, D5, G6

4. A* (without expanded list)

S0(+5), A2(+3), B3(+2), B4(+2), C3(+3), D5(+1), G6(+0)

**C. Choose the most efficient search method that meets the criteria indicated below.
Explain your choice.**

1. You are given a state graph with link costs. The running time of the algorithm should be a function of the number of states in the graph and the algorithm should guarantee that the path with shortest path cost is found.

UCS + expanded list

UCS guarantees shortest paths, expanded list makes sure that the running time depends only on the number of states not the number of paths.

2. You are given a state graph with link costs and consistent heuristic values on the states. The running time of the algorithm should be a function of the number of states in the graph and the algorithm should guarantee that the path with shortest path cost is found.

A* + expanded list

A* with consistent heuristic guarantees shortest paths, expanded list keeps the running time a function of number of states.

3. You are given a state graph with no link costs or heuristic values. The algorithm should find paths to a goal with the least number of states and the space requirements should depend on the depth of the first goal found.

Iterative deepening

Guarantees minimum number of states on path to goal and the memory requirements are determined by the last depth-first search (at the level of the first goal found).

Problem 5 – CSP (12 points)

Assume we have four variables (A, B, C, D) and two values (1, 2). We write variable/value assignments as A1, B2, etc. Assume the only legal values are as listed below:

- A-B: A1-B1, A2-B1, A2-B2
- A-C: A1-C2, A2-C1
- A-D: A2-D2
- B-C: B1-C2, B2-C1
- B-D: B2-D2
- C-D: C1-D1, C1-D2

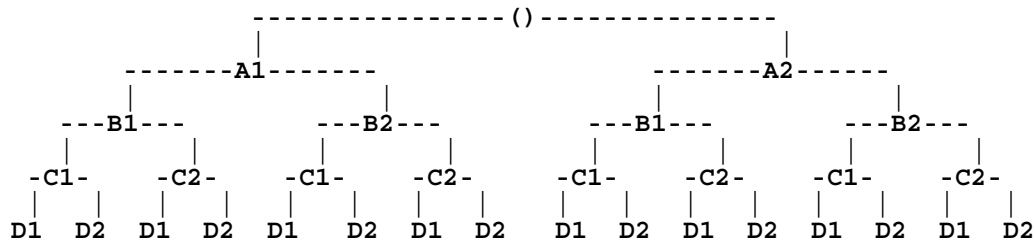
An entry in the matrix below indicates a consistent assignment. This is simply another way of presenting the same information in the list above.

	A1	A2	B1	B2	C1	C2	D1	D2
A1			X			X		
A2			X	X	X			X
B1	X	X				X		
B2		X			X			X
C1		X		X			X	X
C2	X		X					
D1					X			
D2		X		X	X			

Assume you do full constraint propagation in this problem. Show the legal values for each variable after propagation:

- A : A2
- B : B2
- C : C1
- D : D2

Here's the search tree (as in the PS):



Assume that you do the backtracking with forward checking. Show the assignments in order as they are generated during the search.

A1 (FC reduces domain of D to empty, so fail)

A2 (FC reduces domain of C to C1 and domain of D to D2)

B1 (FC reduces domain of D to empty, so fail)

B2 (FC has no further effect)

C1 (FC has no further effect)

D2 (done)

What is the first solution found in the search?

A=2, B=2, C=1, D=2

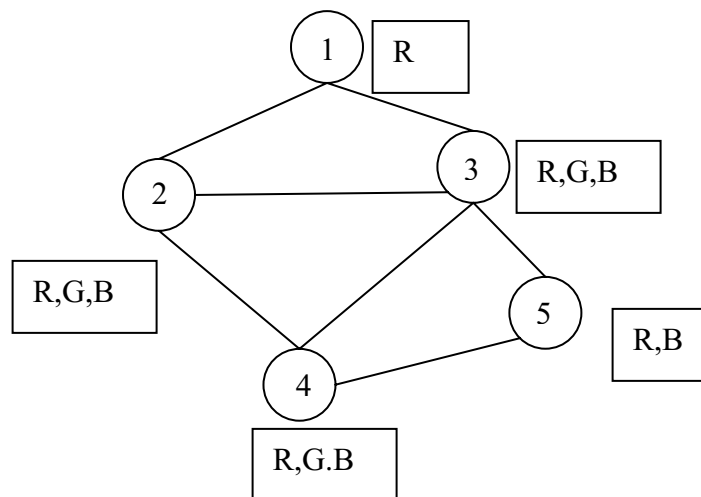
The constraints – repeated for easy reference:

- A-B: A1-B1, A2-B1, A2-B2
- A-C: A1-C2, A2-C1
- A-D: A2-D2
- B-C: B1-C2, B2-C1
- B-D: B2-D2
- C-D: C1-D1, C1-D2

	A1	A2	B1	B2	C1	C2	D1	D2
A1			X			X		
A2			X	X	X			X
B1	X	X				X		
B2		X			X			X
C1		X		X			X	X
C2	X		X					
D1					X			
D2		X		X	X			

Problem 5: CSP (15 points)

Consider the following constraint graph for a graph coloring problem (the constraints indicate that connected nodes cannot have the same color). The domains are shown in the boxes next to each variable node.



1. What are the variable domains after a full constraint propagation?

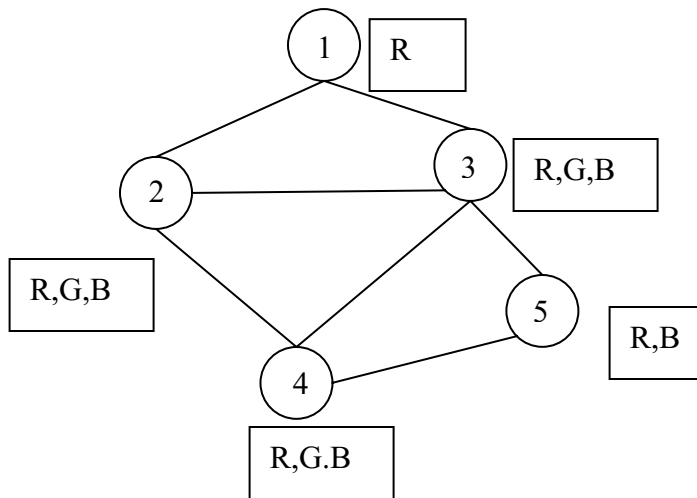
$$1 = \{R\}$$

$$2 = \{G, B\}$$

$$3 = \{G, B\}$$

$$4 = \{R, G, B\}$$

$$5 = \{R, B\}$$



2. Show the sequence of variable assignments during a pure backtracking search (do not assume that the propagation above has been done), assume that the variables are examined in numerical order and the values are assigned in the order shown next to each node. Show assignments by writing the variable number and the value, e.g. 1R. **Don't write more than 10 assignments, even if it would take more to find a consistent answer.**

1R 2R 2G 3R 3G 3B 4R 5R 5B 4G [4B 2B 3R 3G 4R 5R 5B]

3. Show the sequence of variable assignments during backtracking with forward checking, assume that the variables are examined in numerical order and the values are assigned in the order shown next to each node. Show assignments by writing the variable number and the value, e.g. 1R.

1R 2G 3B 4R 2B 3G 4R 5B